

Building a Transformer Language Model from Scratch

CSE 142 Spring 2026

Programming Assignment

CONTENTS

1 Overview	3
2 Required PyTorch Background	3
3 Hard Constraints	4
3.1 Parameter Counting	4
3.2 Architecture Constraints	4
4 BPE Tokenizer (Part 1)	5
4.1 <code>train_bpe(input_path: str, vocab_size: int, special_tokens: list[str] None = None)</code>	5
4.2 <code>BPETokenizer(vocab, merges, special_tokens=None)</code>	5
5 Model Architecture (Part 2)	6
5.1 Components	6
5.2 Forbidden Modifications	8
6 Primitives and Loss (Part 3)	8
6.1 Cross-Entropy Loss (Numerically Stable)	8
7 Training Utilities (Part 4)	9
8 Staff-Controlled Training (Part 5)	10
9 Grading	11
9.1 Components and Weights	11
9.2 Loss Thresholds	11
9.3 Bonus Tiers	12
10 Practical Advice	12
10.1 Suggested Implementation Order	12
10.2 Verifying Your Implementation with the Tests	12
10.3 Quick Architecture Screening	14
10.4 Exploration Dimensions	14
10.5 Known Anti-Patterns	14
10.6 Common Bugs That Cost Hours	14
11 Use of AI Coding Assistants	15
12 What You May Not Use	16
12.1 Allowed PyTorch Surface	16
12.2 Banned Shortcut APIs (enforced by unit tests)	16
12.3 Banned Standard-Library Imports (enforced by unit tests)	17
12.4 Reproducibility Rules (reviewable)	17
13 Commands Reference	17
14 What to Submit	18
14.1 Editable Files	18
14.2 Submission ZIP	18
14.3 Experiment Report	19
15 Project Structure	20

In this assignment you will implement every component of a decoder-only transformer language model from scratch and train it on the [TinyStories](#) dataset. You will build a BPE tokenizer, the full model architecture (embedding, attention, feed-forward, normalization), the loss function, and the training utilities — then tune architecture and hyperparameters to minimize loss under a strict parameter budget.

Dataset. The training corpus is a fixed subset of TinyStories — approximately 49,000 stories (~40 MB of text, ~15 M tokens after byte-level BPE with vocabulary size 512). Run

`scripts/prepare_data.py` once; it downloads the fixed subset and writes `data/train.bin` and `data/val.bin`. Staff retrain on the same split.

1 Overview

The assignment has five parts. Parts 1–4 are implementation tasks graded by unit tests; Part 5 is a training-quality threshold measured on staff-retrained loss.

1. **Part 1 — BPE Tokenizer (24%).** Train a byte-level Byte Pair Encoding tokenizer and implement `encode/decode`.
2. **Part 2 — Transformer Model (33%).** In `transformer_lm/model.py`, implement from scratch: `Linear`, `Embedding`, `RMSNorm`, `scaled_dot_product_attention`, `CausalMultiHeadSelfAttention`, `FeedForward`, `TransformerBlock`, `TransformerLM`.
3. **Part 3 — Primitives & Loss (19%).** In `transformer_lm/nn_utils.py`, implement `softmax`, `silu`, and numerically stable `cross_entropy_loss`. `softmax` and `silu` are used by Part 2, so implement them first.
4. **Part 4 — Training Utilities (19%).** Implement `get_batch` (random batching) and `generate` (autoregressive sampling).
5. **Part 5 — Training-quality gate (5%).** Staff retrain your submitted code/config from scratch and award this 5% based on validation loss. Bonus loss tiers earn extra credit on a hidden test set. Thresholds and definitions live in [Section 9](#).

IMPORTANT

Parts 1–4 are all-or-nothing. The percentages above show relative effort, not partial credit. To earn the full 95%, your submission must pass *all* unit tests, package a valid ZIP, and clear the sanity gate (see [Section 3](#)). Part 5 and bonus tiers are scored independently on staff-retrained loss.

Recommended coding order (different from the grading order above): implement the primitives, model, and loss first. See [Section 10.1](#) for the detailed checklist.

2 Required PyTorch Background

Building a transformer from scratch assumes you are comfortable with PyTorch’s tensor API, broadcasting, and `nn.Module`. If any of the topics below are unfamiliar, read the linked material **before** starting Part 2. Otherwise the time you would have spent on the model goes into chasing shape, dtype, and device errors instead. Lecture and section will not retread this; you read it on your own.

All four are official PyTorch documentation:

- **Tensor basics and broadcasting.** [Learn the Basics](#) (a start-to-finish PyTorch intro) and [Broadcasting Semantics](#) (essential for getting attention and FFN shapes right).

- **Building modules from scratch.** [What is torch.nn really?](#) walks from raw tensor operations up through `nn.Module` — the closest published analogue to what you will do in Part 2.
- **Automatic differentiation.** [Autograd Mechanics](#) and the [Autograd tutorial](#). You do not call `.backward()` yourself, but you must understand which operations preserve the graph and why detaching or in-place writes can silently break training.
- **API references.** [torch.Tensor](#) and [torch.nn](#) — consult these for exact signatures, supported dtypes, and the difference between `view`, `reshape`, `permute`, and `transpose` when a unit test fails on a shape mismatch.

3 Hard Constraints

These are enforced by the autograder. Submissions that violate any constraint will be rejected.

Constraint	Value	Notes
Maximum parameters	500,000	Deduplicated; weight tying counts once
Maximum training steps	5,000	Staff retrains from scratch for exactly 5K steps
Vocabulary size	512	Fixed — must match tokenizer
Context length	256	Fixed — must match evaluation
Sanity gate	best val_loss < 3.0	After 5K steps of staff retraining

The sanity gate exists to catch broken submissions. Any working implementation clears it by a wide margin — see [Section 9](#) for the reference numbers and Part 5 / bonus thresholds. Submissions that modify any file outside the six listed in [Section 14.1](#), or add new files anywhere in the tree, are also rejected.

3.1 Parameter Counting

Parameters are counted on the live model via `named_parameters()`, deduplicated by storage pointer. Weight tying (sharing `lm_head.weight` with `token_emb.weight`) is correctly handled — the shared weight is counted once. Note that **non-RoPE buffers** (`named_buffers()`) also count toward the 500K limit, so you cannot smuggle learnable scales in via `register_buffer`. Buffers owned by the provided `RotaryPositionEmbedding` module (`inv_freq`, `cos_cached`, `sin_cached`) are excluded from the count.

The parameter formula (no bias, with weight tying) is:

$$\text{params} = 512 \times d_{\text{model}} + n_{\text{layers}} \times (4d_{\text{model}}^2 + 3d_{\text{model}}d_{\text{ff}})$$

3.2 Architecture Constraints

- `d_model` must be divisible by `n_heads`.
- $d_{\text{head}} = d_{\text{model}} / n_{\text{heads}}$ must be **even** (required by RoPE).
- All dimensions must be positive integers.

The unit tests require `CausalMultiHeadSelfAttention.__init__` to raise `AssertionError` (use Python's `assert`) when `d_model % n_heads != 0` or `d_head` is odd, and `TransformerLM.forward` to raise `AssertionError` on sequences longer than `context_length`. Non-positive architecture dimensions are caught upstream by config and submission validation; you do not need to assert on them in constructors.

4 BPE Tokenizer (Part 1)

Implement a byte-level Byte Pair Encoding tokenizer (no regex pre-tokenization). Your tokenizer is graded by unit tests only — the training script uses a staff reference tokenizer so every submission trains on the same tokenized data. You can do this part before, after, or in parallel with the rest of the assignment.

4.1

`train_bpe(input_path: str, vocab_size: int, special_tokens: list[str] | None = None)`

Returns `(vocab, merges)` where `vocab: dict[int, bytes]` and `merges: list[tuple[int, int]]`.

ALGORITHM BPE training

1. Read the corpus and convert to a sequence of UTF-8 bytes.
 2. Initialize the base vocabulary: `ID  $i \rightarrow \text{bytes}([i])$` for $i \in \{0, \dots, 255\}$.
 3. Repeat until the vocabulary reaches `vocab_size` (minus space for special tokens):
 1. Count all adjacent pairs of token IDs in the corpus.
 2. Select the most frequent pair. **Tie-breaking:** pick the pair (a, b) with highest count; among ties, pick the lexicographically smallest pair (smallest a , then smallest b).
 3. Create a new token ID for the merged pair and replace every occurrence in the corpus.
 4. Append special tokens (if any) with the highest IDs.
-

4.2 **`BPETokenizer(vocab, merges, special_tokens=None)`**

`encode(text) → list[int]`.

1. Split `text` on special-token boundaries, preserving each special token as one token. If special tokens share prefixes, use longest-match priority.
2. For each non-special segment, encode as UTF-8 bytes — one token per byte.
3. Apply merges in the trained order. For each merge pair, scan left-to-right and replace all *non-overlapping* occurrences before moving to the next merge.
4. *Example:* for pair (a, a) in `aaa`, the leftmost two a 's merge first; the trailing a remains its own token.

decode(ids) → str. Concatenate the byte string for each token ID, then decode as UTF-8 with `errors="replace"` so non-UTF-8 byte sequences become the Unicode replacement character instead of raising.

Special-token IDs. Special tokens (if any) receive the *highest* IDs, assigned in the order they appear in the `special_tokens` argument. They are appended *after* all BPE merges and never participate in pair counting or merging.

5 Model Architecture (Part 2)

You will implement a **pre-LN decoder-only transformer** with the following components:

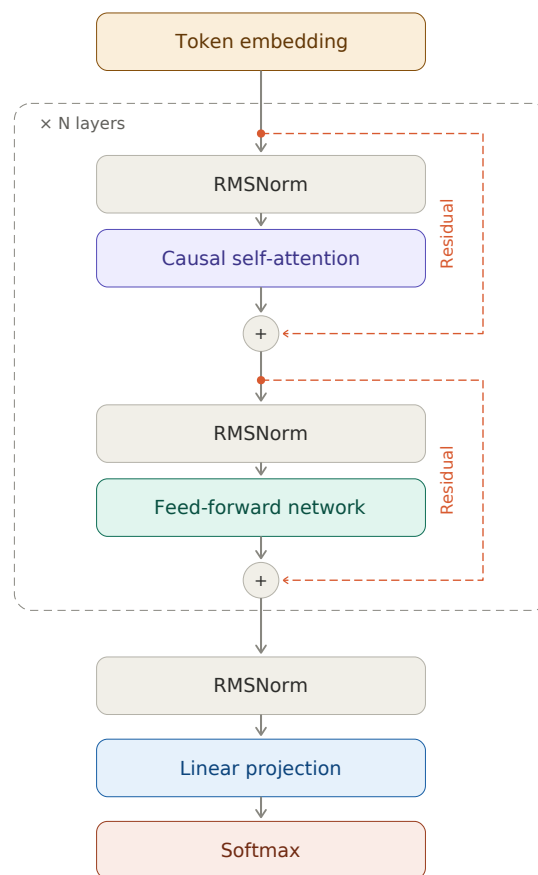


Figure 1: Pre-LN decoder-only transformer architecture.

5.1 Components

Linear. $y = xW^T + b$. Implement `torch.nn.Linear` from scratch using `nn.Parameter`. Initialize weights uniformly in $[-1/\sqrt{d_{\text{in}}}, 1/\sqrt{d_{\text{in}}}]$; initialize bias to zero (if used). Note: the transformer model uses `bias=False` for all projections, but your `Linear` should support optional bias for generality.

Embedding. Look-up table mapping integer indices to dense vectors. Initialize with $\mathcal{N}(0, 0.02)$.

RMSNorm. Root-Mean-Square normalization over the final feature dimension, with **no learnable scale/gain parameter** (unlike many reference implementations). Zero parameters. The constructor takes an `eps` argument; the starter default is 1×10^{-5} .

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\text{mean}_{\text{last dim}}(x^2) + \varepsilon}}$$

Softmax. Signature: `softmax(x: torch.Tensor, dim: int = -1) -> torch.Tensor`, where `dim` selects the axis over which to normalize. Subtract the max along `dim` for numerical stability:

$$\text{softmax}(x_i) = \frac{\exp(x_i - \max(x))}{\sum_j \exp(x_j - \max(x))}$$

You must implement this from scratch — calling `torch.softmax`, `torch.logsumexp`, or any related shortcut is forbidden.

SiLU.

$$\text{silu}(x) = x \cdot \sigma(x)$$

where σ is the logistic sigmoid. Also known as the Swish activation. Used in the SwiGLU feed-forward network.

Attention.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + \text{mask}\right)V$$

The mask is *additive*: positions to attend to get 0; positions to mask out get a large negative value (e.g. -10^9) so they round to 0 after softmax. Shape: broadcastable to $(B, n_{\text{heads}}, T, T)$; the same dtype/device as Q .

RoPE. The `RotaryPositionEmbedding` class is **provided** in `model.py`. Do not modify it. Apply it inside your attention module to Q and K after reshaping into heads (see Causal MHA below).

Causal MHA. Implement the attention module in this order:

1. Apply one **fused QKV projection**, $d_{\text{model}} \rightarrow 3d_{\text{model}}$, no bias.
2. Split into Q, K, V and reshape each to $(B, n_{\text{heads}}, T, d_{\text{head}})$.
3. Apply the provided RoPE module to Q and K .
4. Build an additive causal mask (-10^9 above the diagonal).
5. Compute scaled dot-product attention.
6. Concatenate heads back to (B, T, d_{model}) .
7. Apply the output projection, then **dropout** before the residual add in the block.

FFN (SwiGLU).

$$\text{FFN}(x) = \text{dropout}(W_{\text{down}}(\text{silu}(W_{\text{gate}}x) \odot W_{\text{up}}x))$$

Uses three weight matrices: W_{gate} and W_{up} project from d_{model} to d_{ff} , then W_{down} projects back.

Block (Pre-LN). The forward pass through one block:

$$x = x + \text{MHA}(\text{RMSNorm}(x))$$

$$x = x + \text{FFN}(\text{RMSNorm}(x))$$

TransformerLM. `token_emb` $\rightarrow$ blocks $\times n_{\text{layers}}$ $\rightarrow$ `ln_final` $\rightarrow$ `lm_head`. The output is raw logits of shape (B, T, V) .

IMPORTANT

Two details are easy to miss and will break the unit tests:

Weight tying. `lm_head.weight` must be the *same tensor* as `token_emb.weight` (not a copy). In `TransformerLM.__init__`, set `self.lm_head.weight = self.token_emb.weight`.

Zero-init. In `TransformerLM.__init__`, after creating all blocks, zero each block's `attn.o_proj.weight` and `ffn.w_down.weight`. Do this in `TransformerLM`, *not* inside `TransformerBlock.__init__` — the tests instantiate standalone blocks with normal init and check that the residual path is active.

5.2 Forbidden Modifications

The hidden evaluator reloads your trained weights into a staff-canonical `TransformerLM`. Extra parameters, renamed parameters, or architecture changes outside the required decoder-only transformer (pre-LN, RMSNorm *without* learned gain, RoPE, SwiGLU FFN, weight tying) typically fail strict state-dict loading. Specifically prohibited:

- GQA/MQA (grouped or multi-query attention).
- Mixture of Experts (MoE).
- Layer sharing or parameter tying beyond `lm_head` / `token_emb`.
- Factorized embeddings.
- Learned normalization parameters (RMSNorm gain vectors).
- Additional skip connections, residual scaling, or gating mechanisms.
- Changes to the RoPE implementation.

6 Primitives and Loss (Part 3)

`transformer_lm/nn_utils.py` contains three functions: `softmax`, `silu`, and `cross_entropy_loss`. The first two are specified alongside the model components in [Section 5](#) (Part 2 imports them); the only piece left to specify here is the numerically stable cross-entropy loss. You do **not** implement an optimizer — staff training fixes `torch.optim.AdamW` and the gradient clipper (see [Section 8](#) for the full list of staff-fixed values).

6.1 Cross-Entropy Loss (Numerically Stable)

Signature:

`cross_entropy_loss(logits: torch.Tensor, targets: torch.Tensor) -> torch.Tensor`. `logits` has shape (B, T, V) ; `targets` has shape (B, T) with integer entries in $[0, V)$.

Implement the log-sum-exp trick **from scratch** (do not call `torch.logsumexp`, `F.log_softmax`, `F.cross_entropy`, or any equivalent):

$$\mathcal{L} = \frac{1}{BT} \sum_i \left[\max(\ell_i) + \log \sum_j \exp(\ell_{i,j} - \max(\ell_i)) - \ell_{i,t_i} \right]$$

where $\ell_i \in \mathbb{R}^V$ are logits at position i and t_i is the target token index. Return the mean over all BT positions.

7 Training Utilities (Part 4)

`transformer_lm/training_utils.py` contains two functions used by the staff training and evaluation scripts. Both are tested in isolation by `tests/test_training.py`.

get_batch.

```
get_batch(data: torch.Tensor, batch_size: int, context_length: int,
          device: torch.device | str = "cpu"
          ) -> tuple[torch.Tensor, torch.Tensor]
```

Sample `batch_size` random starting positions uniformly with replacement from $0 \leq \text{start}_i < \text{len}(\text{data}) - \text{context_length}$, then return:

- `x : shape (B, ctx), dtype=long`, with `x[i] = data[start_i : start_i + ctx]`.
- `y : shape (B, ctx), dtype=long`, with `y[i] = data[start_i + 1 : start_i + ctx + 1]` (shifted one position right — next-token targets).

Both tensors must be on `device`. Sampling is with replacement across batches; deterministic seeding is the caller's responsibility. If `len(data) <= context_length`, raise `ValueError` — no valid shifted target window exists.

generate.

```
generate(model: torch.nn.Module, prompt_ids: list[int],
         max_new_tokens: int, temperature: float = 1.0,
         context_length: int | None = None) -> list[int]
```

Autoregressive generation under `torch.no_grad()`. The caller is responsible for setting the model to eval mode before calling `generate` (the staff training script does this around its sample-generation calls).

- Infer the device from the model's parameters. If `context_length` is `None`, use `model.context_length`.
- Feed the *last* `context_length` tokens of the running sequence (truncate the prefix if it grew past `context_length`).
- Take logits at the last position only. Divide by `temperature` (assume `temperature > 0` the staff tests do not exercise `temperature = 0`).

- Apply your own `softmax` (*not* `torch.softmax`), then sample one token with `torch.multinomial`.
- Return a Python `list[int]` containing the original prompt followed by all newly sampled tokens (length `len(prompt_ids) + max_new_tokens`).

8 Staff-Controlled Training (Part 5)

IMPORTANT

You submit code and configuration, not trained checkpoints. Staff retrain from random initialization for exactly 5,000 steps using *only* the six files in [Section 14.1](#), then evaluate on a hidden test set. Your final tier comes from that retraining, not from any number you report locally.

You control architecture and training hyperparameters via `config.yaml`, which the training script reads automatically. The 13 allowed keys, their defaults, and suggested tuning ranges are:

Key	Default	Suggested range
<code>d_model</code>	64	—
<code>n_layers</code>	5	—
<code>n_heads</code>	4	—
<code>d_ff</code>	256	—
<code>dropout</code>	0.0	0.0 to 0.1
<code>learning_rate</code>	10^{-3}	10^{-3} to 10^{-1}
<code>min_lr</code>	10^{-5}	10^{-5} to 10^{-3}
<code>batch_size</code>	32	32 to 1024
<code>warmup_steps</code>	200	100 to 1000
<code>weight_decay</code>	0.1	0.01 to 0.3
<code>gradient_clip_norm</code>	1.0	0.5 to 5.0
<code>beta1</code>	0.9	0.85 to 0.95
<code>beta2</code>	0.95	0.9 to 0.999

The default architecture (`d_model=64` / `n_layers=5` / `n_heads=4` / `d_ff=256`) is approximately 360K parameters, well within the 500K budget but not optimal. All values must be numeric. Unknown keys are **rejected as errors** during staff grading — this prevents typos from silently changing your submission’s behavior.

Staff-fixed values during grading (your submission cannot change these): `vocab_size=512`, `context_length=256`, `max_steps=5000`, `seed=42`, `optimizer = AdamW`. Mixed-precision uses bf16 autocast on CUDA (parameters and AdamW state stay fp32; only forward-pass activations are bf16); CPU/MPS falls back to fp32. **Tier calibration ran under bf16 on CUDA**, so train locally on a CUDA GPU when you can — fp32/CPU results may differ slightly.

No local GPU? Google Colab works. Google Colab no longer offers a free student plan, but the free tier with a single T4 GPU is more than enough for this assignment. One catch: Colab’s local storage is ephemeral and gets wiped when the runtime disconnects, so mount Google Drive and keep your repo, checkpoints, and tokenized data there — otherwise you’ll lose your work between sessions.

The training script exposes `--max_steps` and `--seed` CLI flags for local experimentation (staff grading still uses the fixed values above). CLI arguments override `config.yaml`:

```
PYTHONPATH=. python3 scripts/train.py --max_steps 100 --batch_size 64
```

Customizing the LR schedule. The default in `transformer_lm/lr_schedule.py` is cosine decay with linear warmup. You may rewrite the function body, but the signature **must not change** (the training script calls it once per step):

```
def lr_schedule(step: int, max_steps: int, base_lr: float,
               min_lr: float, warmup_steps: int) -> float:
```

9 Grading

9.1 Components and Weights

Component	Weight	Method
Part 1: BPE tokenizer	24%	Unit tests
Part 2: Transformer model	33%	Unit tests
Part 3: Numerical primitives & loss	19%	Unit tests
Part 4: Training utilities (<code>get_batch</code> , <code>generate</code>)	19%	Unit tests
Part 5: Training-quality gate	5%	Validation-loss threshold
Bonus: Loss optimization	Extra credit	Hidden-test loss tiers

Parts 1–4 base credit is all-or-nothing (see [Section 1](#)): percentages reflect relative effort, not partial credit. Test locally with `pytest tests/ -x -q` and `scripts/sanity_check.py` before submitting.

9.2 Loss Thresholds

The document refers to three different loss numbers; they measure different things on different held-out sets:

Gate	Metric	Threshold	Affects
Sanity gate	best validation loss	< 3.0	Required for Parts 1–4 base credit
Part 5	best validation loss	≤ 1.70	Additional 5%
Bonus tiers	hidden test loss	$\leq 1.53 / 1.47 / 1.43$	Extra credit (see below)

For reference, the random baseline is $\log(512) \approx 6.24$ and the untuned default config converges to about 1.85 validation loss (≈ 1.92 test). Any working implementation clears the sanity gate by a wide margin; reaching the Part 5 threshold takes some genuine tuning of hyperparameters or architecture within the budget, but nothing exotic.

9.3 Bonus Tiers

Bonus Tier	Hidden Test Loss	Demonstrates
Excellent	≤ 1.43	Near-optimal architecture and training
Good	≤ 1.47	Strong architecture and hyperparameter choices
Pass	≤ 1.53	Correct implementation with solid tuning

Tiers were calibrated empirically against the 500K parameter / 5,000 step budget and are final for this offering.

HINT

Interpreting your local numbers. Bonus tiers come from a single staff retrain at `seed=42` on a hidden test set drawn from a held-out TinyStories pool. Your local runs will differ for two reasons: (1) seed noise shifts loss by a non-trivial amount across seeds; (2) hidden test loss is typically a small positive gap above your local validation loss. Treat local numbers as estimates, not predictions of your grade.

10 Practical Advice

10.1 Suggested Implementation Order

The grading parts are numbered 1–5; for the fastest test feedback, implement in this dependency order.

1. `nn_utils.py`: `softmax`, `silu`
2. `model.py` (bottom-up): `Linear` $\rightarrow$ `Embedding` $\rightarrow$ `RMSNorm` $\rightarrow$ `scaled_dot_product_attention` $\rightarrow$ `CausalMultiHeadSelfAttention` $\rightarrow$ `FeedForward` $\rightarrow$ `TransformerBlock` $\rightarrow$ `TransformerLM`
3. `nn_utils.py`: `cross_entropy_loss`
4. `training_utils.py`: `get_batch`, `generate`
5. Sanity check: `python3 scripts/sanity_check.py`
6. Train and tune hyperparameters
7. `tokenizer.py`: BPE tokenizer (independent of the rest)

10.2 Verifying Your Implementation with the Tests

The unit tests are the source of truth for Parts 1–4 implementation correctness. Passing them is necessary for base credit, but the full 95% also requires a valid submission ZIP and a staff retrain that clears the sanity gate (see [Section 3](#) and [Section 9](#)). The suite is organized so you can run

exactly the tests for the component you just finished, which is a much faster loop than running the full suite each time.

Run one model component at a time (Part 2 uses test classes, one per stub):

```
PYTHONPATH=. python3 -m pytest tests/test_model.py::TestLinear -v
PYTHONPATH=. python3 -m pytest tests/test_model.py::TestEmbedding -v
PYTHONPATH=. python3 -m pytest tests/test_model.py::TestRMSNorm -v
PYTHONPATH=. python3 -m pytest tests/
test_model.py::TestScaledDotProductAttention -v
PYTHONPATH=. python3 -m pytest tests/
test_model.py::TestCausalMultiHeadSelfAttention -v
PYTHONPATH=. python3 -m pytest tests/test_model.py::TestFeedForward -v
PYTHONPATH=. python3 -m pytest tests/test_model.py::TestTransformerBlock -v
PYTHONPATH=. python3 -m pytest tests/test_model.py::TestTransformerLM -v
```

Run one function-level test (Parts 1, 3, 4 use function-level tests):

```
PYTHONPATH=. python3 -m pytest tests/
test_nn_utils.py::test_softmax_correctness -v
PYTHONPATH=. python3 -m pytest tests/
test_training.py::test_get_batch_correctness -v
```

Match tests by keyword:

```
PYTHONPATH=. python3 -m pytest tests/ -k "softmax or silu" -v
PYTHONPATH=. python3 -m pytest tests/ -k "tokenizer" -v
```

Run everything one part at a time:

```
PYTHONPATH=. python3 -m pytest tests/test_tokenizer.py -x -q # Part 1
PYTHONPATH=. python3 -m pytest tests/test_model.py -x -q # Part 2
PYTHONPATH=. python3 -m pytest tests/test_nn_utils.py -x -q # Part 3
PYTHONPATH=. python3 -m pytest tests/test_training.py -x -q # Part 4
```

Common flags: drop `-x` to see all failures at once; add `-vv` for full assertion diffs. The `TestAntiCheat` class in `test_model.py` verifies you haven't called banned shortcut APIs from [Section 12](#) — keep those green.

Once the unit suite is fully green, run `scripts/final_check.py` (see [Section 14](#)) before zipping. It runs the structural checks, the full test suite, the sanity overfit, a short training preview on the real data, and a state-dict round-trip, in that order, and stops at the first failure.

HINT

When a test fails, pytest prints the assertion, the actual vs expected tensor, and a traceback. Read the assertion message carefully before diving into a debugger. Most failures point directly at the bug: wrong shape, missing scale, forgotten residual, wrong dtype.

10.3 Quick Architecture Screening

You do not need full 5,000-step training for every architecture. Short screening runs give useful signal:

```
PYTHONPATH=. python3 scripts/train.py --max_steps 500 \
  --d_model 40 --n_layers 4 --n_heads 5 --d_ff 160
```

A 500-step run takes 1–3 minutes on CPU.

10.4 Exploration Dimensions

Within the 500K budget, explore:

- **Width vs. depth:** wider models with fewer layers vs. narrower models with more layers.
- **FFN ratio:** how much budget to allocate to d_{ff} relative to d_{model} .
- **Head count:** more heads with smaller d_{head} vs. fewer heads with larger d_{head} .
- **Learning rate:** the single most impactful training hyperparameter.
- **Batch size:** affects tokens seen per step and optimization dynamics.
- **Dropout:** explore the full range from 0.0 to 0.1. Whether dropout helps depends strongly on the corpus size and training-step budget you actually use; you should measure rather than guess.

10.5 Known Anti-Patterns

- $d_{\text{model}} \leq 16$: performs poorly regardless of depth.
- Very deep narrow models (e.g., $d_{\text{model}}=24$, $n_{\text{layers}}=10$): slow and weak at this parameter scale.
- Single-layer models: too shallow to learn patterns.

10.6 Common Bugs That Cost Hours

When a test fails, check this list before reaching for the debugger. These are the mistakes that historically take students the longest to find:

- **Wrong tensor shape after MHA reshape.** Verify the rearrangement at every step:
 $(B, T, d_{\text{model}}) \rightarrow (B, T, 3d_{\text{model}}) \rightarrow \text{split} \rightarrow (B, n_{\text{heads}}, T, d_{\text{head}}) \rightarrow \text{attention} \rightarrow (B, T, d_{\text{model}})$.
- **RoPE applied before head reshape.** Apply RoPE to Q and K *after* reshaping into heads, not on the flat d_{model} vectors.

- **Multiplicative mask instead of additive.** The mask is *added* to the pre-softmax scores. A multiplicative $\frac{0}{1}$ mask gives nonzero attention to masked positions.
- **Forgetting last-position-only logits in `generate`.** Sample from `logits[:, -1, :]`, not from every position.
- **Cross-entropy reduction mistakes.** Make sure you *mean* over all *BT* positions, not *sum*, and that the log-sum-exp uses `keepdim=True` consistently.
- **Targets not shifted in `get_batch`.** `y[i] = data[start+1 : start+ctx+1]`, not `data[start : start+ctx]`.
- **Weight tying broken by checkpoint round-trip.** Set `self.lm_head.weight = self.token_emb.weight` in `__init__`, not in `forward` — otherwise `load_state_dict` replaces it with a separate tensor.
- **Zero-init in the wrong place.** Zero `o_proj.weight` and `w_down.weight` in `TransformerLM.__init__`, not in `TransformerBlock.__init__` (see [Section 5](#)).
- **Learned RMSNorm gain.** RMSNorm has *no* learnable parameters in this assignment. Adding a `nn.Parameter` gain vector blows the param count and fails tests.
- **Context-length assertion missing.** `TransformerLM.forward` must raise `AssertionError` on sequences longer than `context_length` — and must therefore *store* `self.context_length`.

11 Use of AI Coding Assistants

Use LLMs and coding assistants (ChatGPT, Claude, Copilot, Cursor, etc.) to learn. Ask them to explain a PyTorch API you do not understand, walk you through the math behind RoPE or SwiGLU, tell you why broadcasting applies in a particular shape, or read an error message back at you in plain English. An assistant used as a patient tutor is one of the better ways to build the intuition this assignment exists to teach.

Do not use them to **write the assignment for you**. Specifically:

- Do not paste a stub signature or docstring into an assistant and submit whatever it gives back.
- Do not ask one to write the body of `Linear`, `Embedding`, `RMSNorm`, the attention modules, `FeedForward`, `TransformerBlock`, `TransformerLM`, `softmax`, `silu`, `cross_entropy_loss`, the BPE tokenizer, `get_batch`, `generate`, or `lr_schedule`.
- Do not have an assistant “translate” or “port” a reference implementation (Karpathy’s nanoGPT, Hugging Face Transformers, textbook code) into your submission.
- Do not submit AI-generated code you do not fully understand and could not have written yourself.

The test: could you re-derive what you wrote on a closed-book exam? Code you typed because an assistant explained the underlying concept, and that you then wrote yourself in your own words after closing the chat, is fine. Code you copied from an assistant’s output, even with light edits, is not.

IMPORTANT

Submissions that look LLM-authored will be reviewed for academic integrity. Honest red flags include implementations that mix conventions you have not been taught (e.g., GQA, learned RMSNorm gain, Hugging Face naming), code commented in a voice clearly different from your report, and configurations that copy a known public template verbatim.

Disclosure is required. Your report (see [Section 14](#)) must include a short section describing your use of LLMs and coding assistants — which tools you used, which parts of the assignment you got help on, and how that help shaped what you ended up writing. If you did not use any assistants, say so explicitly. Honest disclosure will not affect your grade; undisclosed use that surfaces during integrity review will.

12 What You May Not Use

In short: implement the neural-network pieces yourself, do not change the submitted file boundary, and do not change the required transformer architecture. The lists below give a high-level summary; the canonical enforced lists live in `tests/test_model.py` (the `TestAntiCheat` class and module-level import scanners). When in doubt, the tests are authoritative.

HINT

Why so restrictive? In real projects you should reach for `torch.nn.Linear`, `F.scaled_dot_product_attention`, and other optimized library kernels. They are banned *here* only to force you to implement the underlying math, which is the whole point of this assignment. After the course, drop the bans and use those kernels in production.

12.1 Allowed PyTorch Surface

- `torch.nn.Parameter`
- Container classes: `Module`, `ModuleList`, `Sequential`
- `torch.nn.Dropout` (not a learned module)
- All standard `torch.Tensor` operations and `torch.autograd`

Used by staff during training (not your choice; listed for transparency): `torch.optim.AdamW` (the only optimizer used during grading) and `torch.nn.utils.clip_grad_norm_` (with `norm_gradient_clip_norm` from `config.yaml`).

12.2 Banned Shortcut APIs (enforced by unit tests)

Tests in `tests/test_model.py` and `tests/test_nn_utils.py` will reject your code before training. Banned by namespace:

- **torch.nn**: Linear, Embedding, LayerNorm, RMSNorm, GELU, SiLU, Softmax, MultiheadAttention.
- **torch.nn.functional (F.)**: softmax, log_softmax, silu, gelu, cross_entropy, nll_loss, embedding, linear, scaled_dot_product_attention, rms_norm.
- **torch / Tensor**: torch.softmax, torch.log_softmax, torch.logsumexp, torch.cross_entropy, torch.nll_loss, Tensor.softmax, Tensor.log_softmax.
- **Private dispatch** (no bypassing the ban): torch._C, torch._C._nn, torch._VF, torch.ops.*.

12.3 Banned Standard-Library Imports (enforced by unit tests)

Inside `model.py`, `nn_utils.py`, `training_utils.py`, and `lr_schedule.py`, do not import: filesystem/process/introspection modules (`os`, `pathlib`, `sys`, `inspect`, `gc`, `subprocess`, `shutil`, `tempfile`, ...), network/serialization modules (`urllib`, `requests`, `http`, `socket`, `pickle`, `marshal`, ...), or built-ins like `open`, `exec`, `eval`, `getattr` on dynamic strings, `__import__`. The full banned list lives in `tests/test_model.py`. `tokenizer.py` is the only submitted module that may `open` the corpus path passed to `train_bpe`.

12.4 Reproducibility Rules (reviewable)

Staff retraining must depend only on your six submitted files and the allowed `config.yaml` keys (Section 8). Specifically forbidden:

- Hard-coded hyperparameters that don't appear in `config.yaml`.
- Pre-trained weights or external data, embedded or downloaded.
- Side channels that change training behavior: wall clock, hostname, GPU identity, environment variables, Python/NumPy unseeded RNG, treating `torch.empty` as zero-initialized.
- Hardware-conditional code paths chosen by introspection (placing tensors on a detected device is fine; *behaviorally* different code paths are not).

Before submitting, retrain locally from scratch with your final `config.yaml` and verify your numbers reproduce; the reported result in your report must come from a run using exactly the submitted config (no CLI overrides). Residual nondeterminism from CUDA bf16 kernels at `seed=42` is small and accepted.

For **banned architecture modifications** (no GQA, no MoE, no learned RMSNorm gain, etc.), see Section 5.2 inside Model Architecture — they live next to the spec they constrain.

13 Commands Reference

```
# Setup (run once)
PYTHONPATH=. python3 scripts/prepare_data.py

# Run unit tests
```

```

PYTHONPATH=. python3 -m pytest tests/ -x -q

# Sanity check (after Parts 2-3)
PYTHONPATH=. python3 scripts/sanity_check.py

# Train with defaults
PYTHONPATH=. python3 scripts/train.py

# Train with custom architecture and hyperparameters
PYTHONPATH=. python3 scripts/train.py --d_model 56 --n_layers 4 \
    --n_heads 4 --d_ff 200 --learning_rate 1e-2 --batch_size 64

# Resume interrupted training
PYTHONPATH=. python3 scripts/train.py --resume

# Final pre-submission check (recommended before zipping)
PYTHONPATH=. python3 scripts/final_check.py [submission.zip]

```

14 What to Submit

14.1 Editable Files

You may **only** edit these six files. The autograder extracts exactly these from your submission ZIP — nothing else is read. You may **not** create new files inside `transformer_lm/` or anywhere else, and you may not modify files in `tests/`, `scripts/`, or any other directory.

File	What you may change
<code>transformer_lm/model.py</code>	Implement all model components (fill in the stubs)
<code>transformer_lm/nn_utils.py</code>	Implement <code>softmax</code> , <code>silu</code> , <code>cross_entropy_loss</code>
<code>transformer_lm/training_utils.py</code>	Implement <code>get_batch</code> and <code>generate</code>
<code>transformer_lm/tokenizer.py</code>	Implement BPE tokenizer
<code>transformer_lm/lr_schedule.py</code>	Customize learning rate schedule (default: cosine with warmup)
<code>config.yaml</code>	Set architecture and training hyperparameters

14.2 Submission ZIP

Submit a single **ZIP file** to Canvas containing exactly these 6 files:

```

transformer_lm/model.py
transformer_lm/nn_utils.py
transformer_lm/training_utils.py
transformer_lm/tokenizer.py
transformer_lm/lr_schedule.py
config.yaml

```

Create the ZIP:

```
zip -r submission.zip transformer_lm/model.py transformer_lm/nn_utils.py \
    transformer_lm/training_utils.py transformer_lm/tokenizer.py \
    transformer_lm/lr_schedule.py config.yaml
```

Validate before uploading. Run `scripts/final_check.py` — it bundles the checks staff grading would catch first into one command:

```
PYTHONPATH=. python3 scripts/final_check.py submission.zip
```

Seven steps in order, stopping at the first failure:

1. ZIP contains all six required files.
2. `config.yaml` parses and the parameter budget checks out.
3. Your Python modules import.
4. `pytest` passes.
5. `sanity_check.py` overfits the tiny batch.
6. A 50-step training preview on the real data shows validation loss going down. (A smoke signal, not a stand-in for the staff retrain or the sanity gate in [Section 3](#).)
7. The model's state dict round-trips through `strict=True` loading with weight tying preserved.

Runtime depends on your config: a few minutes on CPU for default-sized models, longer for larger batches or models near the 500K cap.

For a quick structural check without training or tests, the older `scripts/validate_submission.py submission.zip` runs the first three steps in a few seconds.

14.3 Experiment Report

Upload a **short experiment report** (1–2 pages) separately to Canvas, with two sections:

1. **Experiments and final configuration.** What you tried, what you observed, and why you chose your final setup.
2. **Use of LLMs and coding assistants** (see [Section 11](#)). Required, even if you did not use any. Specify:
 - Which tools you used (e.g., ChatGPT, Claude, Copilot, Cursor) — or state explicitly that you used none.
 - Which parts of the assignment you sought help on (e.g., debugging a shape mismatch in attention, understanding RoPE, interpreting a pytest failure for `cross_entropy_loss`). Be specific — “general help” is not sufficient.
 - How the assistant helped — conceptual explanation, error diagnosis, pointing you at the right PyTorch API, etc. — and how that help shaped what you ended up writing yourself.

Honest disclosure does not affect your grade. The goal is to make your workflow legible; it is also a useful reflective exercise on what you actually learned versus what was handed to you.

15 Project Structure

```
transformer_lm/           # Your code goes here
  tokenizer.py            # Part 1: BPE tokenizer
  model.py               # Part 2: Transformer model components
  nn_utils.py            # Parts 2-3: softmax, SiLU, cross-entropy
  training_utils.py      # Part 4: get_batch, generate
  lr_schedule.py         # LR schedule (customizable)
config.yaml              # Hyperparameter config
tests/
  adapters.py            # Glue code (do not modify)
  test_tokenizer.py      # Part 1 tests
  test_model.py          # Part 2 tests
  test_nn_utils.py       # Part 3 tests
  test_training.py       # Part 4 tests
scripts/
  prepare_data.py        # Data download and tokenization
  train.py               # Training script (reads config.yaml)
  sanity_check.py        # Quick overfit test
  validate_submission.py # ZIP validator
```